

Module 05 Lab - Data Preparation **Objective:** To learn and apply the most common data preparation techniques. Raw data is rarely ready for a machine learning model.

- ✓ This process, also called preprocessing, is one of the most critical steps in the entire ML workflow. **In this lab, you will write more of the code.** Read the explanations and then complete the tasks in the code cells.

- ✓ Part 1: Setup and Initial Look We will continue using the Titanic dataset because it has the exact problems we need to solve: missing values and non-numeric data.

```
import pandas as pd
import numpy as np
# Load the dataset
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')# Let's look at the missing va
print("--- Missing Values Before Cleaning ---")
print(df.isnull().sum())
```

```
--- Missing Values Before Cleaning ---
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age             177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin           687
Embarked         2
dtype: int64
```

Part 2: Handling Missing Values (Imputation) **Concept:** Most machine learning models cannot handle missing values (`NaN`). We must deal with them. Dropping the rows is an option, but you lose data. A better way is **imputation**, which means filling in the missing values with a calculated guess. Common imputation strategies:

- * **Mean:** Fill with the average value. Good for normally distributed data.
- * **Median:** Fill with the middle value. Better for skewed data or data with outliers (like `Fare`).
- * **Mode:** Fill with the most frequent value. Used for categorical data.

- ✓ Task 1: Impute the 'Age' Column The 'Age' column is missing many values. Since age can be skewed (e.g., by a few very old passengers), using the **median** is a robust choice. **Your Task:** Calculate the median of the 'Age' column and use the `.fillna()` method to replace the missing values.

```
# --- ENTER YOUR CODE HERE ---# 1. Calculate the median of the 'Age' column# median_age = ...
#2. Fill the missing values in 'Age' with the median# df['Age'].fillna(..., inplace=True)
# 3. Verify that there are no more missing values in 'Age'# print("Missing values in 'Age' after imputation:")# print(df['Age'])
#Will take the median of the age colloum
median_Age = df['Age'].median()
print(median_Age)
#Will fill all empty age rows with the median
df['Age'] = df['Age'].fillna(median_Age)
print("Missing values in 'Age' after imputation:")
print(df['Age'].isnull().sum())

28.0
Missing values in 'Age' after imputation:
0
```

Part 3: Encoding Categorical Features**Concept:** Machine learning models are mathematical, so they need numbers, not text. We need to convert categorical columns (like 'Sex' and 'Embarked') into a numerical format. The most common method is **One-Hot Encoding**. One-Hot Encoding takes a column with **N** categories and turns it into **N** new columns, each with a **1** or **0**. For example, the 'Sex' column (**male**, **female**) becomes two new columns: **Sex_male** and **Sex_female**. Pandas has a convenient function called `pd.get_dummies()` that does this for us.

Task 2: One-Hot Encode Categorical Columns**Your Task:** Use `pd.get_dummies()` to encode the 'Sex' and 'Embarked' columns. Make sure to drop the original columns after encoding.

```
# --- ENTER YOUR CODE HERE ---
# 1. Use get_dummies to create new columns for 'Sex' and 'Embarked'
# Set 'drop_first=True' to avoid multicollinearity (a statistical issue), which drops one of the new columns (e.g., just having
# encoded_df = pd.get_dummies(df, columns=['Sex', 'Embarked'], drop_first=True)
# 2. Display the first few rows of the new DataFrame to see the new columns
# print(encoded_df.head())
# Will create 2 data rows and drop the original
encoded_df = pd.get_dummies(df, columns=['Sex', 'Embarked'], drop_first=True)
print(encoded_df.head())
```

```
PassengerId  Survived  Pclass \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3
```

```
                                Name  Age  SibSp  Parch \
0                                Braund, Mr. Owen Harris  22.0    1      0
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  38.0    1      0
2                                Heikkinen, Miss. Laina  26.0    0      0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)  35.0    1      0
4                                Allen, Mr. William Henry  35.0    0      0
```

```
                                Ticket    Fare  Cabin  Sex_male  Embarked_Q  Embarked_S
0            A/5 21171    7.2500  NaN      True      False      True
1            PC 17599   71.2833  C85     False      False     False
2  STON/O2. 3101282    7.9250  NaN     False      False     True
3            113803   53.1000  C123     False      False     True
4            373450    8.0500  NaN      True      False     True
```

Part 4: Feature Scaling**Concept:** Many models are sensitive to the scale of the features. For example, **Age** (from 0-80) and **Fare** (from 0-512) are on very different scales. This can cause the model to incorrectly believe that **Fare** is a more important feature simply because its values are larger. **Feature Scaling** solves this by putting all features on a similar scale. A common method is **Standardization** (`StandardScaler` in scikit-learn), which rescales the data to have a mean of 0 and a standard deviation of 1. **Important:** You only scale your numerical features, not your target variable or your newly encoded categorical columns.

Task 3: Scale the 'Age' and 'Fare' Columns**Your Task:** Use `StandardScaler` from `sklearn.preprocessing` to scale the 'Age' and 'Fare' columns.

```
from sklearn.preprocessing import StandardScaler
# --- ENTER YOUR CODE HERE ---
# 1. Create an instance of the StandardScaler# scaler = ...
# 2. Select the columns to scale# columns_to_scale = ['Age', 'Fare']
# 3. Fit the scaler to the data and transform it
# Note: We are using the 'encoded_df' from the previous step if you created it.
# encoded_df[columns_to_scale] = scaler.fit_transform(encoded_df[columns_to_scale])
# 4. Display the first few rows to see the scaled data# print(encoded_df.head())
sc = StandardScaler()
columns_to_scale = ['Age', 'Fare']
#calculates the mean and standard deviation
encoded_df[columns_to_scale] = sc.fit_transform(encoded_df[columns_to_scale])
print(encoded_df.head())
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Name	Age	SibSp	Parch	\
0	Braund, Mr. Owen Harris	-0.565736	1	0	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	0.663861	1	0	
2	Heikkinen, Miss. Laina	-0.258337	0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0.433312	1	0	
4	Allen, Mr. William Henry	0.433312	0	0	

	Ticket	Fare	Cabin	Sex_male	Embarked_Q	Embarked_S
0	A/5 21171	-0.502445	NaN	True	False	True
1	PC 17599	0.786845	C85	False	False	False
2	STON/O2. 3101282	-0.488854	NaN	False	False	True
3	113803	0.420730	C123	False	False	True
4	373450	-0.486337	NaN	True	False	True

 **K** Knowledge Check ** Instructions:** Answer the following questions in this markdown cell.**

1. Why is it often better to impute missing values with the median instead of the mean? It's better to impute missing values with the median because the mean if most data from a set is missing, some of the values can be extremely high or low, so it's better to out whats in the middle.

2. Explain in your own words what One-Hot Encoding does and why it is necessary. One-hot encoding is to take categories and turn them into numbers so the computer can understand. This is necessary so the AI can't get confused while learning and is better than linear encoding where it may correlate 2 groups incorrectly; however, this one is less likely to this because only the value 1 is being used to assign the category.

3. Would you need to apply Feature Scaling to a Decision Tree model? Why or why not? (Hint: Think about how a Decision Tree makes its splits). **[ENTER YOUR ANSWERS HERE]** No you would not need to apply feature Scaling to a Decision tree because the Decision tree only cares about the order not the exact numerical value.

Module 05 Lab - Data Preparation **Objective:** To learn and apply the most common data preparation techniques. Raw data is rarely ready for a machine learning model.

- ✓ This process, also called preprocessing, is one of the most critical steps in the entire ML workflow. **In this lab, you will write more of the code.** Read the explanations and then complete the tasks in the code cells.

Part 1: Setup and Initial Look We will continue using the Titanic dataset because it has the exact problems we need to solve: missing values and non-numeric data.

```
import pandas as pd
import numpy as np

# Load the dataset
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

# Let's look at the missing values
print("--- Missing Values Before Cleaning ---")
print(df.isnull().sum())
```

```
--- Missing Values Before Cleaning ---
PassengerId    0
Survived        0
Pclass         0
Name           0
Sex            0
Age          177
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin        687
Embarked       2
dtype: int64
```

Part 2: Handling Missing Values (Imputation) **Concept:** Most machine learning models cannot handle missing values (NaN). We must deal with them. Dropping the rows is an option, but you lose data. A better way is **imputation**, which means filling in the missing values with a calculated guess. Common imputation strategies:

- * **Mean:** Fill with the average value. Good for normally distributed data.
- * **Median:** Fill with the middle value. Better for skewed data or data with outliers (like Fare).
- * **Mode:** Fill with the most frequent value. Used for categorical data.

Task 1: Impute the 'Age' Column The 'Age' column is missing many values. Since age can be skewed (e.g., by a few very old passengers), using the **median** is a robust choice. **Your Task:** Calculate the median of the 'Age' column and use the `.fillna()` method to replace the missing values.

```
# Calculate the median age
median_age = df['Age'].median()

# Fill missing values in the Age column with the median
df['Age'] = df['Age'].fillna(median_age)

# Check that there are no missing values left
print(df['Age'].isnull().sum())
```

```
0
```

Part 3: Encoding Categorical Features**Concept:** Machine learning models are mathematical, so they need numbers, not text. We need to convert categorical columns (like 'Sex' and

- ✓ 'Embarked') into a numerical format. The most common method is **One-Hot Encoding**. One-Hot Encoding takes a column with **N** categories and turns it into **N** new columns, each with a **1** or **0**. For example, the 'Sex' column (**male**, **female**) becomes two new columns: **Sex_male** and **Sex_female**. Pandas has a convenient function called **pd.get_dummies()** that does this for us.

Task 2: One-Hot Encode Categorical Columns**Your Task:** Use **pd.get_dummies()** to encode the 'Sex' and 'Embarked' columns. Make sure to drop the original columns after encoding.

```
# One-Hot Encode the 'Sex' and 'Embarked' columns
df = pd.get_dummies(df, columns=['Sex', 'Embarked'], drop_first=True)

# Display the first few rows
print(df.head())
```

```
PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3
```

```
      Name  Age  SibSp  Parch  \
0  Braund, Mr. Owen Harris  22.0    1    0
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  38.0    1    0
2  Heikkinen, Miss. Laina  26.0    0    0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)  35.0    1    0
4  Allen, Mr. William Henry  35.0    0    0
```

```
      Ticket  Fare  Cabin  Sex_male  Embarked_Q  Embarked_S
0  A/5 21171  7.2500  NaN      True      False      True
1  PC 17599  71.2833  C85     False      False     False
2  STON/O2. 3101282  7.9250  NaN     False      False     True
3  113803  53.1000  C123     False      False     True
4  373450  8.0500  NaN      True      False     True
```

Double-click (or enter) to edit

Part 4: Feature Scaling**Concept:** Many models are sensitive to the scale of the features. For example, **Age** (from 0-80) and **Fare** (from 0-512) are on very different scales. This can cause the model to incorrectly believe that **Fare** is a more important feature simply because its

- ✓ values are larger. **Feature Scaling** solves this by putting all features on a similar scale. A common method is **Standardization** (**StandardScaler** in scikit-learn), which rescales the data to have a mean of 0 and a standard deviation of 1. **Important:** You only scale your numerical features, not your target variable or your newly encoded categorical columns.

Task 3: Scale the 'Age' and 'Fare' Columns**Your Task:** Use **StandardScaler** from **sklearn.preprocessing** to scale the 'Age' and 'Fare' columns.

```
from sklearn.preprocessing import StandardScaler

# Create a StandardScaler object
scaler = StandardScaler()

# Scale the 'Age' and 'Fare' columns
df[['Age', 'Fare']] = scaler.fit_transform(df[['Age', 'Fare']])

# Display the first few rows
```

```
print(df.head())
```

```

  PassengerId  Survived  Pclass \
0             1         0       3
1             2         1       1
2             3         1       3
3             4         1       1
4             5         0       3

      Name               Age  SibSp  Parch \
0      Braund, Mr. Owen Harris -0.565736    1    0
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  0.663861    1    0
2      Heikkinen, Miss. Laina -0.258337    0    0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)  0.433312    1    0
4      Allen, Mr. William Henry  0.433312    0    0

      Ticket      Fare Cabin  Sex_male  Embarked_Q  Embarked_S
0      A/5 21171 -0.502445   NaN      True      False      True
1      PC 17599  0.786845   C85     False      False      False
2  STON/O2. 3101282 -0.488854   NaN     False      False      True
3      113803  0.420730  C123     False      False      True
4      373450 -0.486337   NaN      True      False      True

```



Instructions: Answer the following questions in this markdown cell.

1. **Why is it often better to impute missing values with the median instead of the mean?** It is often better to use the median because the median is not affected as much by extreme values or outliers. For example, if a few passengers are much older than most others, the mean could be pulled higher, but the median gives a more balanced value.
2. **Explain in your own words what One-Hot Encoding does and why it is necessary.**
3. One-Hot Encoding changes categorical data, like words or labels, into numbers that a machine learning model can understand. It creates separate columns for each category and uses 0s and 1s to show whether each row belongs to that category.
4. **Would you need to apply Feature Scaling to a Decision Tree model?** Why or why not? No, feature scaling is usually not needed for a Decision Tree model. Decision Trees make splits by comparing values in each feature, so they do not depend on the scale of the numbers the same way models like logistic regression or KNN do.

(Hint: Think about how a Decision Tree makes its splits).[ENTER YOUR ANSWERS HERE]

Module 05 Lab - Data Preparation

```
import pandas as pd
import numpy as np

# Load the dataset
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

# Let's look at the missing values
print("--- Missing Values Before Cleaning ---")
print(df.isnull().sum())
```

```
--- Missing Values Before Cleaning ---
PassengerId    0
Survived        0
Pclass         0
Name           0
Sex            0
Age           177
SibSp          0
Parch         0
Ticket         0
Fare           0
Cabin        687
Embarked       2
dtype: int64
```

Calculate the median of the 'Age' column and use the .fillna() method to replace the missing values.

```
# 1. Calculate the median of the 'Age' column
median_age = df['Age'].median()

# 2. Fill the missing values in 'Age' with the median
df['Age'] = df['Age'].fillna(median_age)

# 3. Verify that there are no more missing values in 'Age'
print("Missing values in 'Age' after imputation:")
print(df['Age'].isnull().sum())

print(f"Median age used for imputation: {median_age}")
```

```
Missing values in 'Age' after imputation:
0
Median age used for imputation: 28.0
```

One-Hot Encode Categorical Columns

```
# 1. Use get_dummies to create new columns for 'Sex' and 'Embarked'
# drop_first=True prevents unnecessary duplicate information
encoded_df = pd.get_dummies(df, columns=['Sex', 'Embarked'], drop_first=True)

# 2. Display the first few rows of the new DataFrame to see the new columns
print(encoded_df.head())

print("\nNew columns after encoding:")
print(encoded_df.columns)
```

```
PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3
```

```
          Name  Age  SibSp  Parch  \
0  Braund, Mr. Owen Harris  22.0    1    0
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  38.0    1    0
2  Heikkinen, Miss. Laina  26.0    0    0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)  35.0    1    0
4  Allen, Mr. William Henry  35.0    0    0
```

```
          Ticket  Fare  Cabin  Sex_male  Embarked_Q  Embarked_S
0  A/5 21171  7.2500  NaN      True      False      True
```



```

1      PC 17599  71.2833  C85    False    False    False
2  STON/O2. 3101282  7.9250  NaN    False    False    True
3      113803  53.1000  C123   False    False    True
4      373450  8.0500  NaN     True    False    True

New columns after encoding:
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Age', 'SibSp', 'Parch',
      'Ticket', 'Fare', 'Cabin', 'Sex_male', 'Embarked_Q', 'Embarked_S'],
      dtype='object')

```

Scale the 'Age' and 'Fare' Columns

```

from sklearn.preprocessing import StandardScaler

# 1. Create an instance of the StandardScaler
scaler = StandardScaler()

# 2. Select the columns to scale
columns_to_scale = ['Age', 'Fare']

# 3. Fit the scaler to the data and transform it
encoded_df[columns_to_scale] = scaler.fit_transform(encoded_df[columns_to_scale])

# 4. Display the first few rows to see the scaled data
print(encoded_df.head())

print("\nScaled Age and Fare summary:")
print(encoded_df[columns_to_scale].describe())

```

```

   PassengerId  Survived  Pclass  \
0             1         0       3
1             2         1       1
2             3         1       3
3             4         1       1
4             5         0       3

   Name      Age  SibSp  Parch  \
0  Braund, Mr. Owen Harris -0.565736    1    0
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  0.663861    1    0
2    Heikkinen, Miss. Laina -0.258337    0    0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)  0.433312    1    0
4    Allen, Mr. William Henry  0.433312    0    0

   Ticket      Fare  Cabin  Sex_male  Embarked_Q  Embarked_S
0  A/5 21171 -0.502445   NaN      True      False      True
1    PC 17599  0.786845   C85     False      False      False
2  STON/O2. 3101282 -0.488854   NaN     False      False      True
3    113803  0.420730  C123     False      False      True
4    373450 -0.486337   NaN      True      False      True

Scaled Age and Fare summary:
      Age      Fare
count  8.910000e+02  8.910000e+02
mean   2.272780e-16  3.987333e-18
std    1.000562e+00  1.000562e+00
min   -2.224156e+00 -6.484217e-01
25%   -5.657365e-01 -4.891482e-01
50%   -1.046374e-01 -3.573909e-01
75%    4.333115e-01 -2.424635e-02
max    3.891554e+00  9.667167e+00

```

Knowledge Check

- Why is it often better to impute missing values with the median instead of the mean?
 - Because the mean can be pulled too much by outliers. The median is more stable when the data is skewed or has extreme values.
- Explain in your own words what One-Hot Encoding does and why it is necessary.
 - It turns text categories into number columns with 0s and 1s. It's necessary because machine learning models need numerical input, not words like "male," "female," or "S."
- Would you need to apply Feature Scaling to a Decision Tree model? Why or why not?
 - Decision Trees do not really need feature scaling because they split data based on conditions like "Age < 30" instead of distance or feature size.

